

Reconocimiento de gestos mediante IMU, extracción de características estadísticas y clasificación espacial con k -NN

IABO: Inteligencia Artificial de Borde

Santiago Quintero

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá, Colombia

santiago.quintero-s@mail.escuelaing.edu.co

Juliana Ombita

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá, Colombia

juliana.ombita-c@mail.escuelaing.edu.co

Juan Gómez

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá, Colombia

juan.gfernandez@mail.escuelaing.edu.co

Abstract—En el siguiente documento se presenta el desarrollo e implementación de un flujo de procesamiento (*pipeline*) para el reconocimiento de 5 clases de gestos mediante una Unidad de Medición Inercial (IMU) capturada con un microcontrolador Arduino Nano 33 BLE Sense. Para cada ventana temporal de señal, se extrajeron 6 características estadísticas por cada eje sensorial (media, varianza, curtosis, asimetría, entropía de Shannon y energía), consolidando una matriz de alta dimensionalidad (36 características por muestra). Se aplicó Análisis de Componentes Principales (PCA) para reducir la dimensionalidad a espacios 2D y 3D, evaluando la separación espacial de los grupos. Se compararon dos clasificadores geométricos: uno basado en distancia mínima a centroides (exactitud del 53.33%) y un modelo de k -Vecinos Más Cercanos (k -NN) con ponderación por distancia, el cual alcanzó una exactitud del 80.00% en 2D y del 73.33% en 3D.

Index Terms—IMU, reconocimiento de gestos, PCA, extracción de características, entropía de Shannon, k -NN, Arduino Nano 33 BLE Sense.

I. INTRODUCCIÓN

El análisis de señales inerciales generadas por la aceleración y la velocidad angular permite clasificar patrones de movimiento humano en aplicaciones de interacción persona-computador. En esta tarea, se procesaron señales provenientes de un acelerómetro y giroscopio de 6 ejes (aX, aY, aZ, gX, gY, gZ) divididas en 5 clases de gestos. El objetivo principal radica en extraer descriptores estadísticos relevantes, proyectar el espacio vectorial mediante técnicas de reducción dimensional y evaluar clasificadores capaces de delimitar las fronteras entre gestos.

II. MARCO TEÓRICO

A. Flujo de procesamiento (Pipeline)

Un *pipeline* en aprendizaje automático es una arquitectura modular y determinista formada por fases encadenadas en serie. Cada etapa toma como entrada la salida producida por el

módulo anterior. En este trabajo, el flujo de procesamiento vincula la adquisición inercial, el ventanado temporal, el cálculo de momentos estadísticos, la estandarización, la reducción dimensional y la inferencia espacial.

B. Unidad de Medición Inercial (IMU) y ventanado temporal

Una IMU combina un acelerómetro triaxial $\mathbf{a}(t) = [a_x, a_y, a_z]^T$ y un giroscopio triaxial $\boldsymbol{\omega}(t) = [g_x, g_y, g_z]^T$. Para procesar estas señales continuas, se aplica un esquema de ventanado discreto de longitud $N = 119$ muestras. Sea $x[n]$ la secuencia discreta correspondiente a uno de los 6 ejes en una ventana fija donde $n \in \{1, 2, \dots, N\}$.

C. Descriptores estadísticos en el dominio del tiempo

Para sintetizar el comportamiento dinámico de cada ventana sin perder información relevante, se calculan 6 descriptores formales por cada uno de los 6 ejes sensoriales, totalizando $D = 36$ características por ventana:

- 1) **Media muestral** (μ): Indica el valor central o la componente de corriente continua (DC) de la aceleración o velocidad angular en la ventana:

$$\mu = \frac{1}{N} \sum_{n=1}^N x[n] \quad (1)$$

- 2) **Varianza muestral** (σ^2): Mide el segundo momento central, representando el grado de dispersión o variabilidad de la señal respecto a su media:

$$\sigma^2 = \frac{1}{N-1} \sum_{n=1}^N (x[n] - \mu)^2 \quad (2)$$

- 3) **Coefficiente de asimetría (S):** Tercer momento central estandarizado que evalúa la falta de simetría de la distribución de amplitudes respecto a la media:

$$S = \frac{\frac{1}{N} \sum_{n=1}^N (x[n] - \mu)^3}{\left(\frac{1}{N} \sum_{n=1}^N (x[n] - \mu)^2\right)^{3/2}} \quad (3)$$

- 4) **Curtosis (K):** Cuarto momento central estandarizado. Cuantifica el grado de apuntamiento de la distribución y la susceptibilidad de la señal a presentar valores extremos o picos prominentes:

$$K = \frac{\frac{1}{N} \sum_{n=1}^N (x[n] - \mu)^4}{\left(\frac{1}{N} \sum_{n=1}^N (x[n] - \mu)^2\right)^2} \quad (4)$$

- 5) **Entropía de Shannon (H):** Métrica de la teoría de la información que mide la incertidumbre, la aleatoriedad y la dispersión probabilística en la distribución de amplitudes del eje inercial.
- 6) **Energía de la señal (E):** Cuantifica la magnitud acumulada o potencia total del movimiento realizado durante el gesto en dicha ventana:

$$E = \sum_{n=1}^N |x[n]|^2 \quad (5)$$

D. Cálculo formal de la entropía de Shannon

Para computar la Entropía de Shannon $H(X)$ sobre la secuencia discreta $x[n]$ de N muestras en una ventana temporal, el dominio continuo de amplitudes se discretiza dividiéndolo en B contenedores o intervalos (*bins*) de igual ancho. La probabilidad empírica $P(x_i)$ de que una muestra caiga en el i -ésimo intervalo se define como:

$$P(x_i) = \frac{n_i}{N}, \quad \text{con} \quad \sum_{i=1}^B P(x_i) = 1 \quad (6)$$

donde n_i representa la cantidad de lecturas dentro del intervalo i . La Entropía de Shannon en bits por símbolo se calcula mediante:

$$H(X) = - \sum_{i=1}^B P(x_i) \log_2 P(x_i) \quad (7)$$

en donde se asume $0 \log_2(0) = 0$. Valores elevados de $H(X)$ caracterizan gestos dinámicos con amplia variabilidad e impredecibilidad en el movimiento, mientras que valores bajos corresponden a estados estáticos o movimientos altamente uniformes.

E. Estandarización de datos (normalización Z-score)

Debido a la disparidad de magnitudes entre métricas (p. ej., la energía frente a la media), es indispensable estandarizar la matriz de características $\mathbf{X} \in \mathbb{R}^{M \times D}$. Cada variable j se transforma para tener media cero ($\mu_j = 0$) y desviación estándar unitaria ($\sigma_j = 1$):

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j} \quad (8)$$

F. Análisis de Componentes Principales (PCA)

PCA es un algoritmo de aprendizaje no supervisado que busca proyectar la matriz estandarizada $\mathbf{Z} \in \mathbb{R}^{M \times 36}$ a un subespacio ortogonal de dimensión reducida $k \ll 36$. Calcula la matriz de covarianza $\mathbf{C} = \frac{1}{M-1} \mathbf{Z}^T \mathbf{Z}$ y resuelve el problema de autovalores y autovectores:

$$\mathbf{C} \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad (9)$$

La proporción de varianza explicada acumulada por k componentes ortogonales se define mediante:

$$\text{Varianza explicada} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^D \lambda_j} \quad (10)$$

G. Modelos geométricos de clasificación

- 1) **Clasificador por centroides (mínima distancia):** Para cada clase $c \in \{1, \dots, C\}$, se calcula el prototipo medio μ_c . Una muestra sin etiquetar $\mathbf{z}^*$ se asigna según la menor distancia euclidiana $d(\mathbf{z}^*, \mu_c)$:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{j=1}^k (p_j - q_j)^2} \quad (11)$$

- 2) **k -Vecinos Más Cercanos (k -NN) ponderado:** Identifica los k puntos de entrenamiento más próximos a $\mathbf{z}^*$. Para mitigar empates y dar mayor relevancia a vecinos cercanos, se aplica una ponderación basada en el inverso de la distancia $w_i = \frac{1}{d(\mathbf{z}^*, \mathbf{z}_i) + \epsilon}$, asignando la etiqueta estimada $\hat{y}$ por regla de mayoría ponderada:

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{z}^*)} w_i \cdot \delta(y_i, c) \quad (12)$$

donde $\delta(y_i, c) = 1$ si $y_i = c$ y 0 en otro caso.

III. METODOLOGÍA

A. Adquisición de datos y protocolo de comunicación

La recolección de las señales inerciales se realizó conectando el microcontrolador Arduino Nano 33 BLE Sense a la computadora receptora mediante puerto USB-Serie (*baud rate*: 9600). Se estableció un protocolo de negociación (*handshake*) donde la computadora solicita el inicio de la captura mediante el comando START, y el microcontrolador responde transmitiendo $N = 119$ muestras por ventana temporal para los 6 ejes inerciales (aX, aY, aZ, gX, gY, gZ), finalizando con el marcador END.

El código embebido implementado en C++ para el microcontrolador se presenta en el Código 1.

```
1 #include "Arduino_BMI270_BMM150.h"
2
3 const int NUM_SAMPLES = 119;
4 int samplesRead = 0;
5
6 void setup() {
7   Serial.begin(9600);
8   unsigned long start = millis();
9   while (!Serial && millis() - start < 3000);
10
11   if (!IMU.begin()) {
12     Serial.println("ERROR_IMU");
13   } else {
14     Serial.println("READY");
15   }
16 }
17
```

```

18 void loop() {
19   if (Serial.available()) {
20     String cmd = Serial.readStringUntil('\n');
21     cmd.trim();
22
23     if (cmd == "START") {
24       float aX, aY, aZ, gX, gY, gZ;
25       samplesRead = 0;
26
27       while (samplesRead < NUM_SAMPLES) {
28         if (IMU.accelerationAvailable() && IMU.gyroscopeAvailable()) {
29           IMU.readAcceleration(aX, aY, aZ);
30           IMU.readGyroscope(gX, gY, gZ);
31
32           Serial.print("unlabeled,");
33           Serial.print(aX, 4); Serial.print(",");
34           Serial.print(aY, 4); Serial.print(",");
35           Serial.print(aZ, 4); Serial.print(",");
36           Serial.print(gX, 4); Serial.print(",");
37           Serial.print(gY, 4); Serial.print(",");
38           Serial.println(gZ, 4);
39           samplesRead++;
40         }
41       }
42       Serial.println("END");
43     }
44   }
45 }

```

Listing 1. Firmware embebido en C++ para lectura de IMU y transmisión serie.

Para automatizar la captura de múltiples ventanas por gesto y estructurar el conjunto de datos en archivos CSV (data/raw/gesto_X/ventana_YY.csv), se desarrolló un script en Python (Código 2) que gestiona las señales DTR/RTS del puerto serie y almacena los datos de forma iterativa.

```

1 import serial
2 import time
3 import os
4
5 PUERTO = 'COM8'
6 BAUD_RATE = 9600
7
8 def capturar_dataset():
9     print("=== CAPTURA AUTOMATIZADA DE DATASET ===")
10    num_gesto = int(input("N mero de gesto a grabar (1 al 5): "))
11    total_ventanas = int(input("Ventanas a tomar (10 a 15): "))
12
13    try:
14        arduino = serial.Serial(PUERTO, BAUD_RATE, timeout=1)
15        arduino.dtr = False
16        arduino.rts = False
17        time.sleep(0.5)
18        arduino.dtr = True
19        arduino.rts = True
20        time.sleep(2)
21
22        for v in range(1, total_ventanas + 1):
23            ruta = f"data/raw/gesto_{num_gesto}/ventana_{v:02d}.csv"
24            os.makedirs(os.path.dirname(ruta), exist_ok=True)
25
26            input(f"[Gesto {num_gesto} | Ventana {v}/{total_ventanas}] Presiona ENTER...")
27
28            arduino.reset_input_buffer()
29            arduino.write(b"START\n")
30            arduino.flush()
31
32            with open(ruta, "w") as f:
33                f.write("label,aX,aY,aZ,gX,gY,gZ\n")
34                registros = 0
35                tiempo_inicio = time.time()
36
37                while True:
38                    if arduino.in_waiting > 0:
39                        linea = arduino.readline().decode('utf-8', errors='ignore')
40                        .strip()
41
42                        if linea == "END":
43                            break
44                        elif linea and linea not in ["READY", "ERROR", "ERROR_IMU"]
45                        ]:
46                            f.write(linea + "\n")
47                            registros += 1
48
49                            if time.time() - tiempo_inicio > 3:
50                                arduino.write(b"START\n")
51                                arduino.flush()
52                                tiempo_inicio = time.time()
53
54            except Exception as e:
55                print(f"Error: {e}")
56            finally:
57                if 'arduino' in locals() and arduino.is_open:
58                    arduino.close()
59
60 if __name__ == "__main__":
61     capturar_dataset()

```

Listing 2. Script en Python para automatización de la captura del conjunto de datos.

B. Extracción de características y reducción (PCA)

Se aplicaron las ecuaciones de los 6 descriptores estadísticos sobre las ventanas de 119 muestras generadas, construyendo la matriz de $M \times 36$. Luego de normalizar los valores vía Z-score, se aplicó PCA obteniendo:

- Espacio 2D (PC1 y PC2): Explicación del 45.27% de la varianza total (véase la Fig. 1).
- Espacio 3D (PC1, PC2 y PC3): Explicación del 53.78% de la varianza total (véase la Fig. 2).

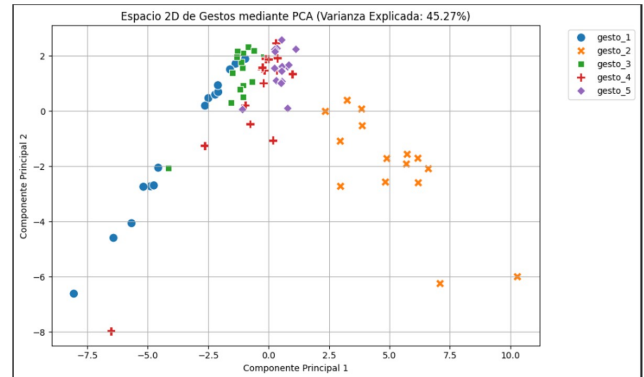


Fig. 1. Proyección en espacio 2D de las muestras de gestos mediante PCA (Varianza explicada: 45.27%).

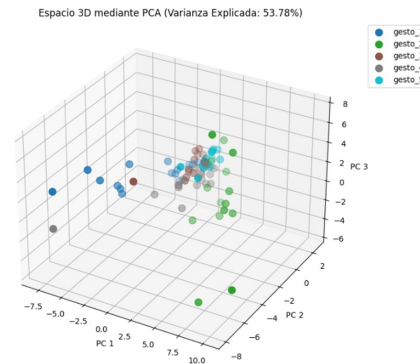


Fig. 2. Proyección tridimensional (3D) de los gestos mediante PCA (Varianza explicada: 53.78%).

IV. CLASIFICACIÓN Y RESULTADOS

Se dividió el conjunto de datos estandarizado en 80% entrenamiento y 20% prueba (*Train/Test Split*). Se evaluaron los dos enfoques geométricos presentados formalmente:

A. Clasificador geométrico por centroides

Al evaluar la distancia mínima a prototipos estáticos, se evidenció que la superposición física entre los gestos 3, 4 y 5 degrada el rendimiento del modelo lineal, obteniendo una exactitud global limitada de 53.33%.

B. Clasificador por k -Vecinos Más Cercanos (k -NN)

Mediante un modelo k -NN ($k = 3$) con ponderación por distancia inversa:

- En el subespacio **2D**, las fronteras de decisión no lineales adaptativas permitieron delimitar adecuadamente las clases, alcanzando una exactitud del 80.00% (véase la Fig. 3).
- En el subespacio **3D**, la delimitación de volúmenes de decisión alcanzó una exactitud del 73.33% (véase la Fig. 4).

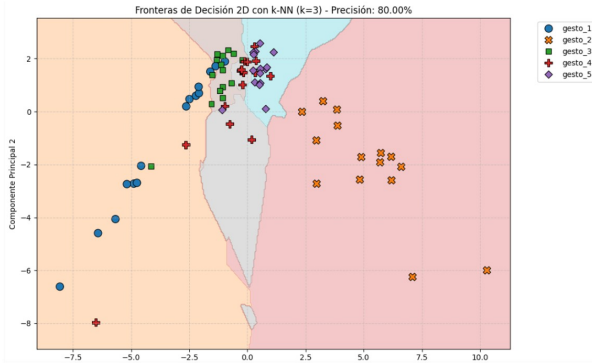


Fig. 3. Fronteras de decisión en espacio 2D con el clasificador k -NN ($k = 3$), alcanzando una exactitud del 80.00%.

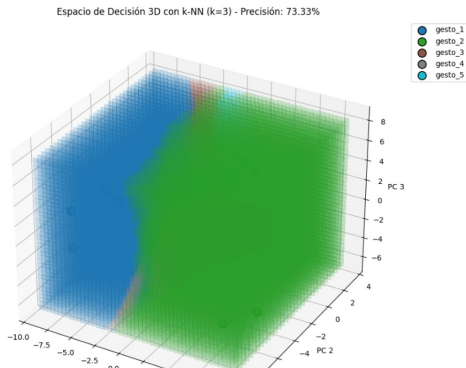


Fig. 4. Espacio de decisión tridimensional (3D) generado por k -NN ($k = 3$), alcanzando una exactitud del 73.33%.

TABLE I
COMPARATIVA DE CLASIFICACIÓN SEGÚN EL MODELO Y LA DIMENSIONALIDAD

Modelo	Espacio	Varianza expl.	Exactitud
Centroides	2D	45.27%	53.33%
k -NN ($k = 3$)	2D	45.27%	80.00%
k -NN ($k = 3$)	3D	53.78%	73.33%

V. CONCLUSIONES

1. La formulación matemática explícita de descriptores como la Entropía de Shannon y la Curtosis permitió empaquetar variaciones temporales complejas en vectores de

características altamente representativos. 2. La reducción dimensional por PCA redujo el costo computacional para la ejecución de clasificadores, logrando una diferenciación clara con hasta un 80.00% de acierto al emplear k -NN no lineal.

VI. DISPONIBILIDAD DE CÓDIGO Y RECURSOS

Con el propósito de permitir la verificación, replicación y evaluación interactiva de los procedimientos experimentales, el código ejecutable y los repositorios asociados se encuentran disponibles en los siguientes enlaces de acceso público:

- **Cuaderno de trabajo en Google Colab (procesamiento y modelado):**

<https://colab.research.google.com/drive/1MwRyJxjTTYbNNvpelzPtUa6F1DZ13h51?usp=sharing>

- **Repositorio de código fuente y conjunto de datos completo (GitHub):**

https://github.com/santiagoquinterosolanoelectronic123/Creaci-n-de-Dataset_IMU_IABO_SQS_JG_JO